

Saul Alexander Sanchez Bautista

Tecnologías utilizadas

Backend y Servidor: El sistema utiliza **Node.js** junto con el framework **Express** para gestionar las rutas de la API, servir los archivos estáticos y procesar las peticiones HTTP. También emplea los módulos nativos `fs` y `path` para la lectura de archivos locales y `dotenv` para proteger las variables de entorno.

Inteligencia Artificial: Se integra el SDK oficial **`@google/genai`** conectado al modelo **`gemini-3.5-flash-lite`**, elegido por su baja latencia y su amplia ventana de contexto.

Frontend e Interfaz: La interfaz web está desarrollada con **HTML5**, **CSS3** (utilizando variables personalizadas, Flexbox y transiciones por hardware) y **JavaScript** puro (ES5/ES6) para la manipulación del DOM y peticiones asíncronas mediante `Fetch API (AJAX)`. Se complementa con **Tailwind CSS** para utilidades rápidas, **Font Awesome** para la iconografía vectorial, **Web Speech API** para el reconocimiento de voz y **LocalStorage** para la persistencia de datos en el navegador.

Control de Versiones y Despliegue: El código fuente se gestiona en **GitHub** y se despliega en la nube mediante la plataforma **Render**, configurada con puertos dinámicos para entornos de producción, y además hay una versión funcional web actualmente al que ahora mismo puedes acceder desde aquí.

Arquitectura del sistema

Patrón Cliente-Servidor Monolítico: El proyecto sigue una arquitectura desacoplada a nivel lógico pero alojada en un mismo repositorio, dividida en una capa de presentación (`public/`) y una capa de lógica de negocio (`server/`).

Capa de Presentación (Cliente): El navegador carga la interfaz gráfica, gestiona el estado visual (tema claro/oscuro, accesibilidad de texto y barra lateral) y almacena el historial de conversación en el dispositivo del usuario mediante `localStorage`.

Capa de Procesamiento (Servidor): Actúa como un intermediario seguro (*proxy*) entre el usuario y los servicios de Google. El servidor oculta la clave privada de la API (`GEMINI_API_KEY`), expone los endpoints `/api/chat` y `/api/sugerencias`, e inyecta el contexto documental antes de consultar al modelo.

Forma en que se integró la inteligencia artificial

Conexión mediante SDK Oficial: Se inicializó una instancia de `GoogleGenAI` en el servidor (`server/conex.js`) autenticada mediante variables de entorno seguras.

Ingeniería de Prompts e Instrucciones del Sistema: Se configuró la propiedad `systemInstruction` para definir el rol estricto del asistente. Esto obliga al modelo a actuar únicamente como un consultor de la UASD y le prohíbe inventar información externa al documento provisto.

Gestión de Memoria Conversacional: El frontend envía tanto la consulta actual como el arreglo acumulado de mensajes previos (`history`). El servidor transforma estos datos al formato de roles alternos (`user` y `model`) que exige Gemini, permitiendo respuestas coherentes en preguntas de seguimiento.

Tolerancia a Fallos: Se implementó un bloque de captura de errores (`try-catch`) que activa un sistema de respaldo local con respuestas predefinidas sobre temas clave (misión, claustro y autonomía) en caso de interrupciones de red o saturación de la API.

Funcionamiento del chatbot

Inicio y Carga de Sugerencias: Al abrir la aplicación, el cliente solicita al endpoint `/api/sugerencias` el listado de preguntas frecuentes extraídas del archivo `questions.txt` y las renderiza en un carrusel horizontal.

Entrada de Consulta: El usuario puede escribir su pregunta, seleccionar una sugerencia del carrusel o utilizar el micrófono integrado, el cual transcribe la voz a texto y envía la consulta automáticamente al finalizar.

Procesamiento y Respuesta: El cliente muestra un indicador animado de carga y envía la petición `POST` a `/api/chat`. El servidor combina el historial, la nueva pregunta y el estatuto, consulta a Gemini con una temperatura baja (`0.2`) para priorizar la precisión factual, y devuelve la respuesta.

Renderizado y Persistencia: La interfaz muestra la respuesta en un globo de chat estilo mensajería instantánea y guarda ambos turnos en `localStorage`, actualizando al mismo tiempo la barra lateral de navegación rápida.

Uso del Estatuto Orgánico como fuente de información

Inyección de Contexto Completo (Long-Context): Al arrancar el servidor, Node.js lee el archivo plano `server/data/statute.txt` y lo carga en memoria. En

cada petición al chat, el texto íntegro del Estatuto Orgánico se inyecta dentro de las instrucciones del sistema del modelo.

Ventaja sobre Búsquedas Tradicionales: Aprovechando la capacidad de procesamiento de contexto masivo de `gemini-3.5-flash-lite` (hasta un millón de tokens), el chatbot analiza la totalidad del reglamento universitario en tiempo real sin necesidad de dividir el texto en fragmentos ni usar bases de datos vectoriales externas.

Control de Alucinaciones: El prompt del sistema establece una restricción explícita que limita la base de conocimiento del bot exclusivamente al contenido de `statute.txt`. Si un usuario consulta sobre un tema ajeno al estatuto, el asistente responde amablemente que dicha información no se encuentra en el documento oficial.